

# Massive Parallel Computing with GPUs

## PDE Simulation of Bullet Option

Elio Moreau   Hugo Negrel

Sorbonne Université

March 29, 2026



# Outline

Context

Monte-Carlo algorithms on device

PDE resolution of Bullet option

# Background to Financial Engineering

It is assumed that the time evolution law of the asset price follows the Black-Scholes model. For  $0 \leq s \leq t \leq T$ :

$$S_t = S_s \exp((r - \sigma^2/2)(t - s) + \sigma\sqrt{t - s}G), \quad (1)$$

where  $r$  is the risk-free rate,  $\sigma$  is the SDE's volatility and  $G$  is a random variable following the centered standard normal law  $N(0, 1)$ .

It is often required in finance to price options. It involves estimating quantities of the form:

$$F(t, T, S_t, r, \sigma, K) = \mathbb{E}[f((S_u)_{u \in [t, T]}, K)],$$

where  $K$  is called the strike of the option and  $f$  the pay-off. Example:

- ▶ European Call:  $F(t, T, S_t, r, \sigma, K) = e^{-r(T-t)}\mathbb{E}[(S_T - K)_+ | S_t]$ .
- ▶ Americal Call:  $F(t, T, S_t, r, \sigma, K) = \sup_{\tau \in [t, T]} \mathbb{E}[e^{-r(\tau-t)}(S_\tau - K)_+ | S_t]$

# Bullet Option

This project aims to pricing an exotic contract called the *bullet option*. Bullet option requires discretizing the time interval into  $M + 1$  sub-interval with an uniformly spaced time schedule  $T_0 = 0 < T_1 < T_2 < \dots < T_M < T_{M+1} = T$ , so that  $\forall i \in \{1, \dots, M + 1\}$ ,  $T_i = \frac{i}{M+1} T = i\Delta t$ ,  $\Delta t = \frac{i}{M+1}$ .

The price of a bullet option is<sup>1</sup>:

$$F(t, S_t, j) = e^{-r(T-t)} \mathbb{E}_{(S_u)_{u \in [t, T]}} [X | S_t = x, I_t = j], \text{ with} \\ X = (S_T - K)_+ 1_{I_T \in [P_1, P_2]}, \text{ and define for all } t \in [0, T], I_t = \sum_{T_i \leq t} 1_{S_{T_i} < B}.$$

Note that by construction,  $I_t \leq \#\{T_i \leq t\}$ , i.e.  $j$  must take value in  $\{0, 1, \dots, i\}$ . The variable  $B$  is commonly called the barrier.

---

<sup>1</sup>Since  $\sigma, r$  and  $T$  are fixed, they will be ignored from now on. 

# Pricing Bullet option with Monte-Carlo

As  $I_T$  depends on the path history at instants  $(T_i)_i$ , simulating directly  $S_T$  from  $S_{T_i}$  only is insufficient for computing  $X$ ; the asset price at each intermediate times  $(T_i)_{i \in \{1, \dots, M+1\}}$  must be simulated.

- 1:  $T_i = \frac{i}{M+1} T$ ,  $I_{T_i} = j$
- 2: **for**  $m = 1$  to  $\text{Nb}_{\text{traj}}$  **do**
- 3:   **for**  $k = i$  to  $N$  **do**
- 4:      $G \sim N(0, 1)$
- 5:      $S_{T_{k+1}} = S_{T_k} \exp((r - \sigma^2/2)(T_{k+1} - T_k) + \sigma \sqrt{T_{k+1} - T_k} G)$
- 6:      $I_{T_{k+1}} = I_{T_k} + 1_{S_{T_{k+1}} \leq B}$
- 7:   **end for**
- 8:    $X_m = (S_T - K)_+ 1_{I_T \in [P_1, P_2]}$
- 9: **end for**

$$F(T_i, x, j) = \frac{e^{-r(T-T_i)}}{\text{Nb}_{\text{traj}}} \sum_{m=1}^{\text{Nb}_{\text{traj}}} X_m$$

**Algorithm 1:** Pseudocode for the Monte-Carlo calculation of  $F(t, S_{T_i}, j)$

# On-device Algorithms

I propose two implementations:

- ▶ Fix  $T_i, S_{T_i}, I_{T_i}$  and parallelize over trajectories for an estimate of  $F(T_i, S_{T_i}, I_{T_i})$ . Three variants:
  - ▶ Compute the arithmetic mean on the host.
  - ▶ Thread reduction with shared memory
  - ▶ Thread reduction with registers

In this approach, an appropriate pair  $(S_{T_i}, I_{T_i})$  is assumed known. This is a hypothesis that is not part of the exercise. The solution: **sample them prior to the MC estimate.**

- ▶ Given  $T_i$ , and starting from  $S_0$ , sample  $(S_{T_i}, I_{T_i})$ . For each sample, compute a Monte-Carlo estimate of  $F(T_i, S_{T_i}, I_{T_i})$ . Two variants:
  - ▶ Parallelise over 2D grids. The estimate is obtained via a sequential *for* loop, where each thread sampled its own pair  $(S_{T_i}, I_{T_i})$ . Heavy and imbalanced load on threads: **Very bad idea!**
  - ▶ Parallelise over a 3D grids, partitioned as follows:  $T_i$  is spread over  $x$ -axis,  $(S_{T_i}, I_{T_i})$  over  $(x, y)$  plan, and Monte-Carlo trajectories are spread over  $z$ -axis. Threads are then reduced on shared memory along the  $z$ -axis.

# Code time!

## Numerical results: Execution time I

| $\sigma$ | $r$ | $K$ | $S_0$ | $T$ | $M$ | $B$ | $P_1$ | $P_2$ |
|----------|-----|-----|-------|-----|-----|-----|-------|-------|
| .2       | .1  | 100 | 100   | 1   | 100 | 110 | 10    | 40    |

Table 1: Numerical parameters<sup>2</sup> of the bullet option

NB = 128 and NTPB = 512.

| $T_i$ | ms   |
|-------|------|
| 0.01  | 4.79 |
| 0.30  | 4.09 |
| 0.60  | 3.78 |
| 0.90  | 4.39 |
| 1.00  | 3.86 |

(a) No reduction

| $T_i$ | ms   |
|-------|------|
| 0.00  | 3.56 |
| 0.30  | 3.97 |
| 0.60  | 3.80 |
| 0.90  | 4.37 |
| 1.00  | 3.77 |

(b) Reduction w/ s.m

| $T_i$ | ms   |
|-------|------|
| 0.00  | 4.21 |
| 0.30  | 3.66 |
| 0.60  | 3.30 |
| 0.90  | 3.25 |
| 1.00  | 3.76 |

(c) Reduction w/ registers

Table 2: Execution time for the Monte-Carlo estimate of a single  $F$

No big difference between the three approaches.

<sup>2</sup>They are fixed throughout all the presentation



## Numerical results: Execution time II

Grids =  $(M + 1, N, W)$ , NTPB = 512. Each run samples  $N$  pair of values  $(S_{T_i}, I_{T_i})$ . Each tuple  $(T_i, S_{T_i}, I_{T_i})$  has  $\text{NTPB} \times W$  threads to estimate  $F$ .

| N    | Execution time (ms) |
|------|---------------------|
| 128  | 2797                |
| 256  | 5596                |
| 512  | 10668               |
| 1024 | 21463               |

(a)  $W = 128$

| N    | Execution time (ms) |
|------|---------------------|
| 128  | 5533                |
| 256  | 11097               |
| 512  | 22781               |
| 1024 | 48128               |

(b)  $W = 256$

**Table 3:** Time execution for the Monte-Carlo estimate on a grid also sampled by Monte-Carlo.

## Feynman-Kac formula

Feynman-Kac formula establishes a clear, modern link between parabolic partial differential equations (PDEs) and stochastic differential equations (SDEs).

Let  $u(t, x, j) = e^{r(T-t)}F(t, e^x, j)$  where  $F$  is the price of a bullet option as before. One can then show that, on any interval  $t \in [T_{M-k}, T_{M-k+1})$ ,  $k = M, \dots, 0$ ,  $u(t, x, j)$  is the solution of the backward parabolic PDE

$$\frac{\partial u}{\partial t}(t, x, j) + \mu \frac{\partial u}{\partial x}(t, x, j) + \frac{1}{2}\sigma^2 \frac{\partial^2 u}{\partial x^2}(t, x, j) = 0$$

with

$$\mu = r - \frac{\sigma^2}{2}.$$

The final and boundary conditions are:

$$\begin{aligned} & - u(T, x, j) = \max(e^x - K, 0) \mathbf{1}_{\{j \in [P_1, P_2]\}} \text{ for any } (x, j) - \\ & u(t, \log[2K/5], j) = p_{\min} = 0 - u(t, \log[5K/2], j) = p_{\max} = \frac{3K}{2} \mathbf{1}_{\{j \in [P_1, P_2]\}} \end{aligned}$$

# Crank Nicolson method

The Crank Nicolson scheme is a semi implicit scheme to solve linear equations. Once the  $x$  and  $t$  coordinates are discretised, labeling at fixed  $j$   $u(t_k, x_i, j) = u_i^k$ , it's about solving the linear system:

$$q_u u_{i+1}^k + q_m u_i^k + q_d u_{i-1}^k = p_u u_{i+1}^{k+1} + p_m u_i^{k+1} + p_d u_{i-1}^{k+1}$$

For our equation:

$$q_u = -\frac{\mu \Delta t}{4 \Delta x} - \frac{\sigma^2 \Delta t}{4 \Delta x^2}, \quad q_d = \frac{\mu \Delta t}{4 \Delta x} - \frac{\sigma^2 \Delta t}{4 \Delta x^2}, \quad q_m = 1 + \frac{\sigma^2 \Delta t}{2 \Delta x^2}$$

$$p_u = \frac{\mu \Delta t}{4 \Delta x} + \frac{\sigma^2 \Delta t}{4 \Delta x^2}, \quad p_d = -\frac{\mu \Delta t}{4 \Delta x} + \frac{\sigma^2 \Delta t}{4 \Delta x^2}, \quad p_m = 1 - \frac{\sigma^2 \Delta t}{2 \Delta x^2}$$

## Numerical results: Execution time

Here  $NB=64$  is used for the different values of  $j$  while NTPB is used for the grid on the  $x$ -axis. Here we see the execution time for different values of NTPB and different values of  $N_t$  the number of timepoints between  $T_M$  and  $T$ .

| $N_t$ | ms   |
|-------|------|
| 10    | 0.56 |
| 50    | 0.88 |
| 100   | 1.57 |
| 500   | 4.58 |
| 1000  | 8.53 |

(a) NTPB = 256

| $N_t$ | ms    |
|-------|-------|
| 10    | 0.56  |
| 50    | 1.22  |
| 100   | 1.57  |
| 500   | 6.02  |
| 1000  | 12.26 |

(b) NTPB = 512

| $N_t$ | ms    |
|-------|-------|
| 10    | 0.83  |
| 50    | 2.13  |
| 100   | 3.79  |
| 500   | 18.09 |
| 1000  | 34.85 |

(c) NTPB = 1024

## Discontinuities

To solve the PDE at all times  $\in [T_0, T)$  we have to address the discontinuities at the times  $T_k$  for  $k \in 0, \dots, M$

$$u_{T_{M-k}}(S_{T_{M-k}}, j) = \begin{cases} u_{T_{M-k}}(S_{T_{M-k}}, P_2) \mathbf{1}_{\{S_{T_{M-k}} \geq B\}}, & \text{if } j = P_2, \\ u_{T_{M-k}}(S_{T_{M-k}}, P_k^1) \mathbf{1}_{\{S_{T_{M-k}} < B\}}, & \text{if } j = P_k^1 - 1, \\ \begin{pmatrix} u_{T_{M-k}}(S_{T_{M-k}}, j) \mathbf{1}_{\{S_{T_{M-k}} \geq B\}} + \\ u_{T_{M-k}}(S_{T_{M-k}}, j+1) \mathbf{1}_{\{S_{T_{M-k}} < B\}} \end{pmatrix} & \text{if } j \in [P_k^1, P_2 - 1]. \end{cases}$$

Where  $P_k^1 = \max(P_1 - k, 0)$ . When solving the PDE over  $[T_0, T_{M+1}]$ , figures on tables of the previous slide are all on average multiplied by 100, hinting an execution time of the order of the second.

Finite-Difference:  $F(T_0, S_0, 0) = 6.5$ . Monte-Carlo:  $F(T_0, S_0, 0) = 7.62$

# Code time!

# Conclusion

Each method have its drawbacks and advantages. What comes first from the comparison is that: **Finite difference seems a little bit faster than Monte-Carlo algorithm on 1D problems.** However, several remarks:

- ▶ In Monte-Carlo: no arbitrary grid is imposed during resolution. Synchronization intervenes only at the very end to sum the samples of each trajectory (one per thread). Convergence speed scale as  $\sqrt{N}$ , regardless of the dimension of the space.
- ▶ In finite difference: Convergence and scheme's stability don't scale well in the high dimensional setting; CFL condition becomes more and more restrictive as  $D$  grows. The load of computations mechanically increases. Also, Crank-Nicolson algorithm requires more frequent thread synchronization during the execution, which impedes thread parallelism.

Thanks for You Attention!